

Football Player Management System

1. Context

A football club needs a simple computer-based system to manage players, training sessions, match records, salary calculation, and basic performance reports. The current manual process makes it difficult to track player profiles, training attendance, match statistics, and monthly salary bonuses.

The system stores information such as Player ID, full name, age, nationality, position, shirt number, base salary, player type, status, training sessions, attendance records, match records, and performance records. It must enforce key business rules such as unique IDs, valid shirt numbers, valid dates, attendance overwrite rules, and salary calculation based on a selected month and year.

To keep the project level comparable to a standard PRO192 final project, all players are considered members of the same football club/team. Students are not required to implement Team Management, player assignment, team capacity, team removal, or team-based shirt-number checking.

The system is a Java console application. It must not use a database, GUI, web application, or external framework. ***Data must be saved and loaded using files.***

2. Users

Football Club Staff - manages players, training sessions, match records, salary calculation, and reports.

3. Functional Requirements

Player Management

1. Add a new player with Player ID, full name, age, nationality, position, shirt number, base salary, player type, and status.
2. Update player information such as position, shirt number, base salary, and status. The player type cannot be changed after the player is created.
3. Deactivate a player by updating the player's status to Inactive instead of permanently deleting player data.
4. View all players.
5. Search players by name, position, nationality, or status.
6. Display detailed information of a selected player.

Training and Match Management

7. Create a training session with Training ID, date, location, and topic.
8. Record training attendance by entering absent Player IDs. When attendance is recorded for the first time, all active players at that time are included in the attendance record. Players not listed as absent are marked as present.
9. Create a match record with Match ID, date, opponent team, and match type. The opponent team is stored as text only.

10. Add or update player performance in a match, including goals, assists, yellow cards, red cards, and minutes played.
11. View training history.
12. View match history.

Contract and Salary Management

13. Display player type, such as Regular Player or Star Player, for salary calculation.
14. Calculate monthly salary for a selected month and year.
15. Calculate performance bonus for eligible players.
16. Display salary summary of all players.
17. Validate contract and salary rules before calculation.

Reporting

18. Generate Salary Summary report for a selected month and year.
19. Generate All-time Top Goal Scorers report from all existing match performance records.

4. Business Rules

BR1: Player ID, Training ID, and Match ID must be unique and cannot be modified after creation.

BR2: Player full name, position, nationality, base salary, and shirt number must not be empty or invalid.

BR3: Position must be one of: Goalkeeper, Defender, Midfielder, or Forward.

BR4: Player age must be between 16 and 45. For simplicity, Age is stored as an integer in this project; Date of Birth is not required.

BR5: Shirt number must be between 1 and 99.

BR6: Two active players cannot have the same shirt number.

BR7: When updating a player from Inactive to Active, the system must validate that the player's shirt number does not duplicate the shirt number of any other active player.

BR8: Player status must be either Active or Inactive.

BR9: Only players with status Active can be included in new training attendance and match performance records.

BR10: The system does not support physical deletion of any player to maintain historical data integrity. To remove a player from active use, the user must update the player's status to Inactive.

BR11: Base salary must be greater than zero.

BR12: Dates must follow the format dd/MM/yyyy and must be valid calendar dates.

BR13: Month must be between 1 and 12. The year must be between 2000 and 2100.

BR14: Match type must be one of: Friendly, League, or Cup.

BR15: Absent Player IDs must belong to existing active players when attendance is first recorded. Duplicate absent IDs are not allowed.

BR16: Attendance is first created based on the list of active players at the time attendance is saved.

BR17: Attendance for a training session can be recorded only once. If recorded again, the system must overwrite the present/absent status instead of creating a duplicate attendance record.

BR18: When updating an existing attendance record, the system must use the player list already stored in the original attendance snapshot. It must not rebuild the attendance list from the current active player list.

BR19: Goals, assists, yellow cards, red cards, and minutes played must not be negative.

BR20: Minutes played must be between 0 and 120.

BR21: If minutes played is 0, goals, assists, yellow cards, and red cards must all be 0.

BR22: A player can have only one performance record per match.

BR23: If the selected player does not have a performance record in the selected match, the system creates a new performance record. If the record already exists, the system must display the old data and ask for confirmation before replacing it.

BR24: $\text{Performance Points} = \text{goals} \times 5 + \text{assists} \times 3 - \text{yellowCards} \times 1 - \text{redCards} \times 3$.
If the result is negative, use 0.

BR25: Salary calculation must be based on a selected month and year. Monthly performance points are summed from all match records of that player within the selected month and year.

BR26: Regular Player bonus = 0 VND. Star Player bonus = Monthly Performance Points x 500,000 VND.

BR27: $\text{Total Monthly Salary} = \text{Base Salary} + \text{Monthly Performance Bonus}$.

BR28: The required All-time Top Goal Scorers report is calculated from all existing match performance records and is not filtered by month or year.

BR29: All inputs must be validated before processing. Invalid input must be handled using exception handling and clear error messages.

5. OOP Requirements

- ✓ Encapsulation: All model classes must use private fields and provide appropriate constructors, getters, setters, and business methods.
- ✓ Inheritance: The system must include Player as a superclass and RegularPlayer and StarPlayer as subclasses.
- ✓ Polymorphism: RegularPlayer and StarPlayer must override a meaningful method, such as `calculateBonus(int monthlyPerformancePoints)` or `calculateMonthlySalary(int monthlyPerformancePoints)`.
- ✓ Abstraction: Use abstract classes or abstract methods where appropriate, for example defining common salary-related behavior in Player.
- ✓ Interfaces: Interfaces may be used where appropriate to define common behaviors, but they are not mandatory if the design is already clear and meaningful.
- ✓ Responsibility design: PerformanceRecord should provide `calculatePerformancePoints()`. Player / RegularPlayer / StarPlayer should calculate bonus and monthly salary.
- ✓ Collections: Use ArrayList, List, HashMap, or Map to store players, training sessions, match records, attendance records, and performance records.
- ✓ Exception handling: Handle invalid input, duplicated IDs, missing records, invalid numbers, invalid dates, and file I/O errors.
- ✓ File I/O: Save and load data using files. Students may use text files, CSV files, or object serialization, but submitted data files must be readable and included in the project folder.

- ✓ Class diagram before coding: Students must draw a class diagram showing classes, attributes, methods, inheritance, associations, and abstraction/interface usage where applicable before implementation.

Suggested Model Classes

- ✓ Player, RegularPlayer, StarPlayer
- ✓ TrainingSession
- ✓ MatchRecord
- ✓ AttendanceRecord
- ✓ PerformanceRecord

Suggested manager classes may include PlayerManager, TrainingManager, MatchManager, SalaryManager, and ReportManager. This is only a design suggestion, not a required package structure.

6. Console UI Requirements

- ✓ The program must provide a menu-based console interface.
- ✓ The main modules must include Manage Players, Training and Match Management, Contract and Salary Management, Reports, and Exit.
- ✓ The Reports menu must include Salary Summary and All-time Top Goal Scorers.
- ✓ The system must validate user input and business rules before processing.
- ✓ The system must display clear Success / Fail messages for each task.
- ✓ The system must use console I/O and file I/O only.
- ✓ In task screens, "Submit" means the data is accepted after validation and stored in the program's in-memory collections. Data is written to files when the user chooses "Save and Exit".
- ✓ The sample screens in the SYSTEM INTERFACE section are examples. Students must implement all functional requirements, even if some screens are not shown in the sample interface section.

7. SYSTEM INTERFACE

Note: When invalid input is entered, the system should display a clear error message and prompt the user to re-enter the data without crashing.

1. MAIN MENU

```
=====
FOOTBALL PLAYER MANAGEMENT SYSTEM
=====
1. Manage Players
2. Training and Match Management
3. Contract and Salary Management
4. Reports
5. Exit
-----
Choose an option: _
```

Task S1 - Add Player

```
----- ADD PLAYER -----
Player ID: P01
Full Name: Nguyen Van Minh
Age: 22
Nationality: Vietnam
Position: Forward
Shirt Number: 10
Base Salary: 25000000
Player Type:
1. Regular Player
2. Star Player
Choose Player Type: 2
Status (Active/Inactive): Active
[1] Submit [2] Cancel
Choose an option: 1
Output:
Players added successfully.
```

Task S2 - Update Player

```
----- UPDATE PLAYER -----
Enter Player ID: P01
Current Information:
Name: Nguyen Van Minh | Position: Forward | No.10
Base Salary: 25000000 | Type: Star Player | Status: Active
Enter new Position: Forward
Enter new Shirt Number: 9
Enter new Base Salary: 28000000
Enter new Status (Active/Inactive): Active
[1] Update [2] Cancel
Choose an option: 1
Output:
The player updated successfully.
Note: Updating status to Inactive is used to deactivate a player. Player Type
cannot be changed after creation. If a player is reactivated, the system must
check shirt number duplication again before saving.
```

Task S3 - View All Players

```
----- PLAYER LIST -----
```

ID	Name	Age	Position	Shirt	Type	Status
P01	Nguyen Van Minh	22	Forward	9	Star Player	Active
P02	Tran Quoc Bao	24	Defender	5	Regular Player	Active

```
-----
Press ENTER to return...
```

Task S4 - Search Players

----- SEARCH PLAYERS -----

Search by:

1. Name
2. Position
3. Nationality
4. Status

Choose search type: 2

Enter keyword: Forward

Search Results:

ID	Name	Age	Position	Shirt	Type	Status
P01	Nguyen Van Minh	22	Forward	9	Star Player	Active

Press ENTER to return...

Task S5 - View Player Details

----- PLAYER DETAILS -----

Enter Player ID: P01

Player ID: P01

Full Name: Nguyen Van Minh

Age: 22

Nationality: Vietnam

Position: Forward

Shirt Number: 9

Base Salary: 28000000

Player Type: Star Player

Status: Active

Press ENTER to return...

Task S6 - Create Training Session

----- CREATE TRAINING SESSION -----

Training ID: TR01

Date: 15/12/2026

Location: Club Training Ground

Training Topic: Attacking and Finishing

[1] Submit [2] Cancel

Choose an option: 1

Output:

Training session created successfully.

Task S7 - Record Training Attendance

```
----- RECORD TRAINING ATTENDANCE -----
Training ID: TR01
Date: 15/12/2026
Total Active Players: 2
Enter absent Player IDs, separated by commas.
Leave blank if all included active players are present.
Absent Player IDs: P02
[1] Submit Attendance [2] Cancel
Choose an option: 1
Output:
Training attendance was recorded successfully.
Summary:
Present: 1
Absent: 1
Note: If this attendance is updated later, use the same player list from the
original attendance snapshot.
```

Task S8 - Create Match Record

```
----- CREATE MATCH RECORD -----
Match ID: M01
Date: 20/12/2026
Opponent Team: Hanoi Eagles
Match Type:
1. Friendly
2. League
3. Cup
Choose Match Type: 2
[1] Submit [2] Cancel
Choose an option: 1
Output:
Match record created successfully.
Note: Opponent Team is stored as text only.
```

Task S9 - View Training History

```
----- TRAINING HISTORY -----
ID      Date      Location      Topic
-----
TR01    15/12/2026    Club Training Ground    Attacking and Finishing
-----
Press ENTER to return...
```

Task S10 - View Match History

```
----- MATCH HISTORY -----
ID      Date      Opponent Team    Match Type
-----
M01     20/12/2026    Hanoi Eagles     League
-----
Press ENTER to return...
```

Task S11 - Add / Update Player Performance

----- ADD / UPDATE PLAYER PERFORMANCE -----

Match ID: M01
Player ID: P01
Player Name: Nguyen Van Minh

Case 1 - No existing performance record:
The system allows the user to enter performance data directly.

Case 2 - Existing performance record found:
Existing Performance:
Goals: 1 | Assists: 0 | Yellow Cards: 0
Red Cards: 0 | Minutes Played: 80
Do you want to replace this record?
[1] Replace [2] Cancel
Choose an option: 1

Enter new performance data:
Goals: 2
Assists: 1
Yellow Cards: 0
Red Cards: 0
Minutes Played: 90
[1] Submit [2] Cancel
Choose an option: 1

Output:
Player performance saved successfully.
Performance Points: 13

Note: If no performance record exists, skip the replacement confirmation and enter performance data directly. If a performance record already exists, display the old data and ask for confirmation before replacing it.

Task S12 - Calculate Player Salary

----- CALCULATE PLAYER SALARY -----

Enter Month: 12
Enter Year: 2026
Enter Player ID: P01
Player: Nguyen Van Minh
Type: Star Player
Base Salary: 28000000
Monthly Performance Points: 13
Output:
Salary Summary:
Base Salary: 28000000 VND
Performance Bonus: 6500000 VND
Total Salary: 34500000 VND

Task S13 - Salary Summary Report

```
----- SALARY SUMMARY REPORT -----
Month: 12/2026
ID      Name                Type                Base Salary  Bonus    Total
-----
P01  Nguyen Van Minh    Star Player      28000000    6500000  34500000
P02  Tran Quoc Bao      Regular Player   18000000     0        18000000
-----

Total Monthly Salary Cost: 52500000 VND
Press ENTER to return...
Note: Report samples may assume that more records already exist in the saved
data.
```

Task S14 - All-time Top Goal Scorers

```
----- ALL-TIME TOP GOAL SCORERS -----
Rank  Player ID  Name                Position  Goals
-----
1      P01      Nguyen Van Minh    Forward   8
2      P02      Tran Quoc Bao      Defender  1
-----

Press ENTER to return...
```

Task S15 - Exit Program

```
----- EXIT SYSTEM -----
Do you want to save all data before exiting?
1. Save and Exit
2. Exit without Saving
3. Cancel
Choose an option: 1
Output:
Data saved successfully.
Thank you for using the Football Player Management System.
```

8. Assessment Criteria

- ✓ Correct implementation of required functional modules.
- ✓ Appropriate use of OOP principles: encapsulation, inheritance, polymorphism, abstraction, and interfaces where applicable.
- ✓ Correct use of collections.
- ✓ Correct file I/O and persistent data loading.
- ✓ Input validation and exception handling.
- ✓ Accurate implementation of business rules.
- ✓ Clear console menu and user interaction.
- ✓ Correct and readable required reports.
- ✓ Readable source code and meaningful class design.
- ✓ No database, GUI, web interface, or unrelated external systems.